

Documentation Technique

Vite & Gourmand

Application web de commande en ligne — Traiteur Bordelais

FRAMEWORK	Symfony 8.1 / PHP 8.4
BASE DE DONNÉES	MySQL 9 + MongoDB 7
DÉPLOIEMENT	https://ecf-vg.fly.dev
AUTEUR	Fabrice Olivier
DATE	Juillet 2026

Sommaire

1.	Réflexions initiales technologiques	3
2.	Configuration de l'environnement de travail	4
3.	Modèle Conceptuel de Données (MCD)	5
4.	Diagramme d'utilisation & de séquence	6
5.	Documentation du déploiement	7

1. Réflexions initiales technologiques

Avant de démarrer le développement, j'ai réfléchi aux technologies les plus adaptées au projet Vite & Gourmand. L'objectif était de choisir un stack technique stable, maintenable et cohérent avec les exigences du marché professionnel français.

Symfony 8.1 / PHP 8.4	Framework PHP
Symfony s'impose comme le standard en entreprise en France. Son architecture MVC, ses composants natifs (Security, Mailer, Form, Doctrine) et sa documentation exhaustive en font un choix naturel pour un projet de cette envergure. La version 8.1 apporte les attributs PHP natifs pour les annotations Doctrine, rendant le code plus lisible.	
Doctrine ORM	Mapping objet-relationnel
Intégré à Symfony, Doctrine permet de manipuler la base de données via des objets PHP sans écrire de SQL brut. Il gère les migrations, les relations entre entités (OneToMany, ManyToMany) et garantit la sécurité des requêtes par paramétrage automatique.	
MySQL 9	Base de données relationnelle
La structure des données du projet — commandes, menus, utilisateurs, plats — est naturellement relationnelle. MySQL est supporté nativement par Doctrine et WAMP, et reste le SGBD le plus répandu en production PHP.	
MongoDB 7	Base de données non relationnelle
J'ai choisi MongoDB pour stocker les données analytiques (nombre de commandes par menu, chiffre d'affaires). Sa flexibilité de schéma est adaptée à des agrégations statistiques qui évoluent, sans alourdir le schéma MySQL principal.	
Bootstrap 5	Framework CSS
Bootstrap offre un système de grille responsive, des composants accessibles (attributs ARIA intégrés) et le support natif du mode sombre via data-bs-theme. Il accélère l'intégration tout en respectant les critères RGAA.	
Twig	Moteur de templates
Twig est le moteur officiel de Symfony. Il échappe automatiquement toutes les variables affichées, protégeant contre les attaques XSS. Son système d'héritage de templates évite la duplication du layout.	
Symfony Mailer + Mailtrap	Emails automatiques
Le composant Mailer de Symfony gère l'envoi des emails (confirmation commande, bienvenue, relance). Mailtrap est utilisé en développement pour intercepter les emails sans les envoyer réellement.	

2. Configuration de l'environnement de travail

Prérequis installés

Outil	Version	Rôle
WAMP (Windows)	3.x	Serveur Apache + PHP + MySQL intégrés
PHP	8.4	Moteur d'exécution requis par Symfony 8.1
Composer	2.x	Gestionnaire de dépendances PHP
MongoDB Community	7.x	Base de données pour les statistiques
Git	2.x	Versionnement du code source

Étapes d'installation

1. Cloner le dépôt

```
git clone https://github.com/biofa3/ecf-vg.git
```

2. Installer les dépendances

```
composer install
```

3. Copier le fichier d'environnement

```
cp .env .env.local
```

4. Importer la base MySQL

```
mysql -u root ecf_vg < ecf_vg.sql
```

5. Démarrer MongoDB

```
mongod --dbpath "C:\data\db"
```

6. Lancer Symfony

```
symfony server:start
```

7. Vider le cache

```
php bin/console cache:clear
```

Variables d'environnement (.env.local)

Les informations sensibles sont stockées dans .env.local, exclu du dépôt Git. Ce fichier configure quatre éléments essentiels :

DATABASE_URL	Connexion MySQL locale (ecf_vg)
MONGODB_URI	Connexion MongoDB (localhost:27017)
MAILER_DSN	Sandbox Mailtrap pour les emails en développement
APP_SECRET	Clé secrète Symfony (générée via secret:generate)

3. Modèle Conceptuel de Données

Le schéma ci-dessous représente l'ensemble des entités Doctrine du projet et leurs relations. La base MySQL principale compte 12 entités. Les données analytiques (statistiques, chiffre d'affaires) sont stockées séparément dans MongoDB.

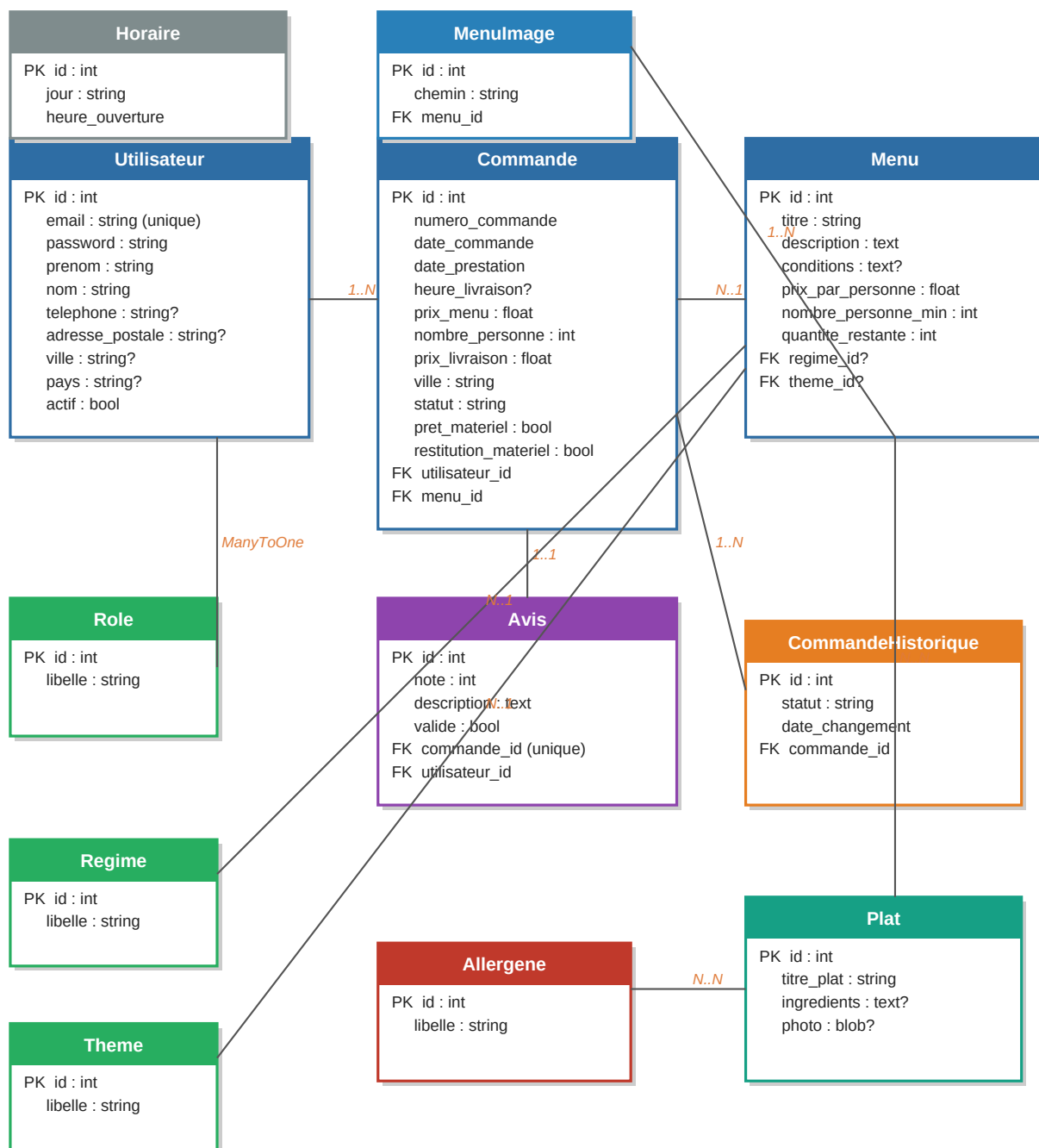


Figure 1 — Modèle Conceptuel de Données (entités Doctrine / MySQL)

Principales relations

Relation	Type	Description
----------	------	-------------

Utilisateur → Commande	OneToMany	Un utilisateur peut passer plusieurs commandes
Commande → Menu	ManyToOne	Chaque commande porte sur un seul menu
Menu → Plat	ManyToMany	Un menu contient plusieurs plats, un plat peut être dans plusieurs menus
Commande → Avis	OneToOne	Un avis est lié à une unique commande livrée
Utilisateur → Role	ManyToOne	Chaque utilisateur a un seul rôle (Admin, Employé, User)
Plat → Allergene	ManyToMany	Un plat peut contenir plusieurs allergènes
Menu → Regime / Theme	ManyToOne	Chaque menu peut avoir un régime et un thème
Commande → CommandeHistorique	OneToMany	Chaque changement de statut est archivé

4. Diagramme d'utilisation & de séquence

4.1 Diagramme d'utilisation (Use Case)

Le diagramme suivant représente les interactions entre les quatre types d'acteurs et les fonctionnalités de l'application.

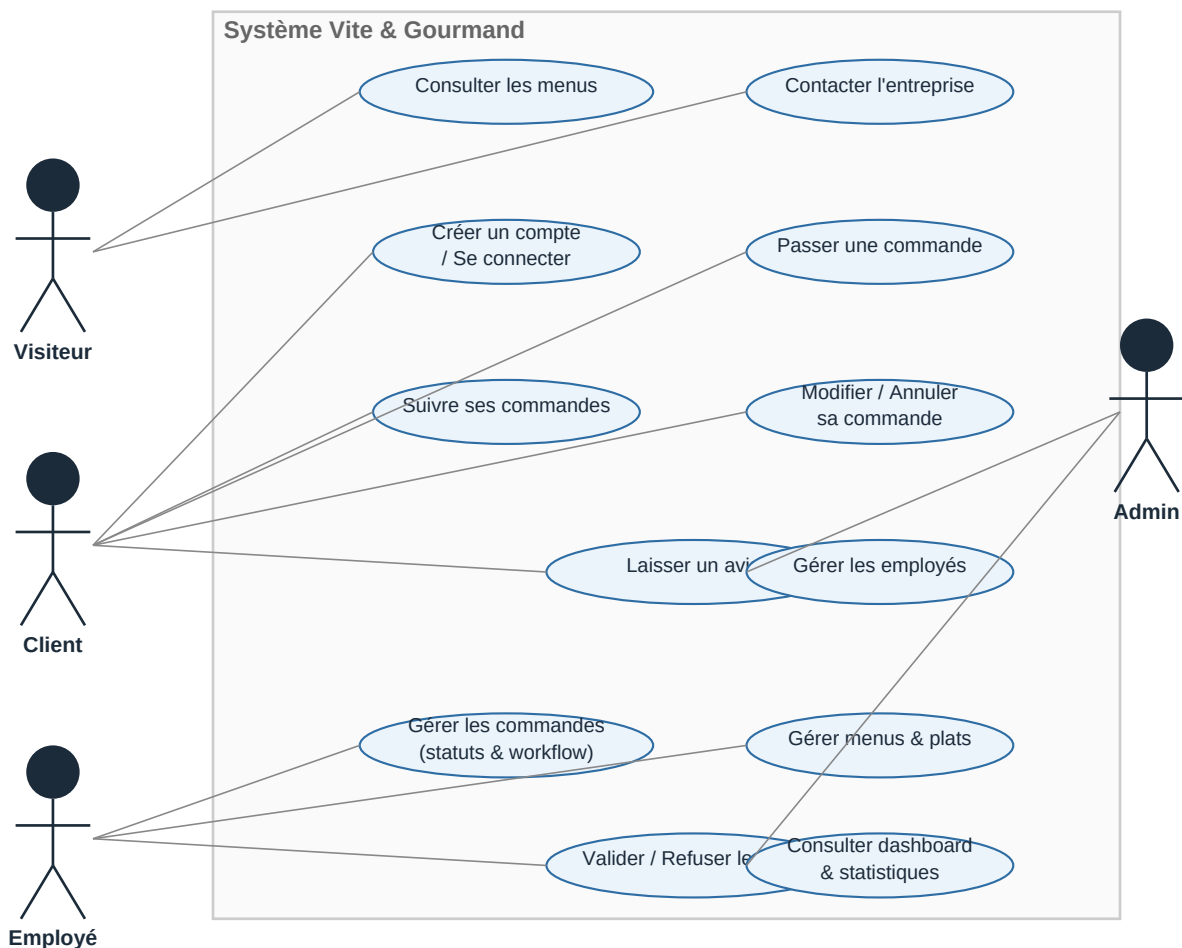


Figure 2 — Diagramme d'utilisation

4.2 Diagramme de séquence — Passer une commande

Le diagramme ci-dessous illustre le flux complet d'une commande : depuis la consultation du menu par le client jusqu'à la persistance en base de données.

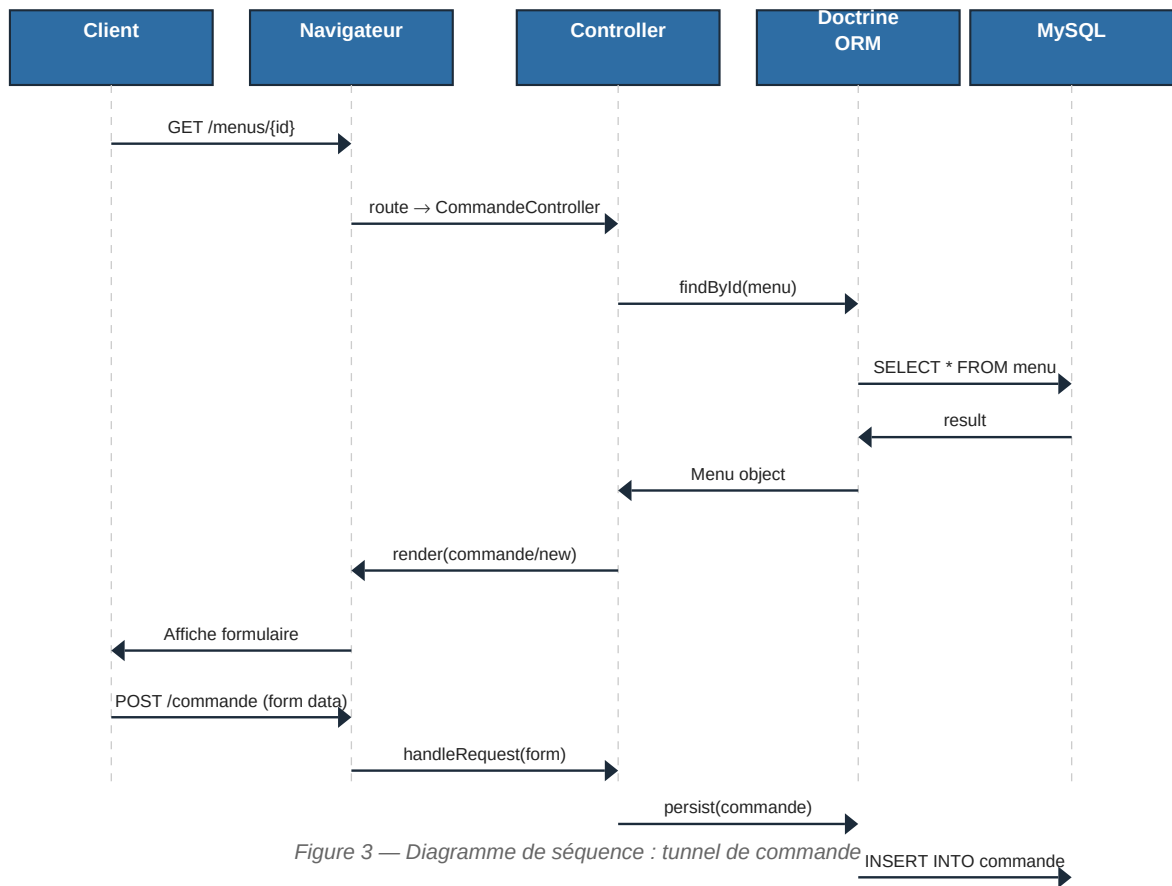


Figure 3 — Diagramme de séquence : tunnel de commande

5. Documentation du déploiement

L'application est déployée en production sur la plateforme fly.io, accessible à l'adresse <https://ecf-vg.fly.dev>. Fly.io a été retenu pour sa simplicité de configuration, son support natif de Docker, et son offre gratuite suffisante pour un projet de cette échelle.

Infrastructure de production

Composant	Service	Détail
Application PHP/Symfony	fly.io (machine virtuelle)	Instance Fly Machine — région Europe (Paris)
Base de données MySQL	Clever Cloud (MySQL add-on)	Base MySQL hébergée sur Clever Cloud — connexion via DATABASE_URL
Base MongoDB	MongoDB Atlas (Free Tier)	Cluster partagé — connexion via MONGODB_URI
Emails	Mailtrap (production)	SMTP sécurisé — envoi réel en production
Assets (CSS/JS/images)	Volume Fly.io	Fichiers statiques servis via Apache

Étapes de déploiement

1. Installation de Fly CLI	Téléchargement et installation de l'outil en ligne de commande flyctl.
2. Connexion au compte	fly auth login — authentification via le compte fly.io.
3. Initialisation de l'application	fly launch — génération automatique du fichier fly.toml décrivant la configuration de déploiement (région, port, ressources).
4. Configuration des variables d'environnement	fly secrets set APP_SECRET=... DATABASE_URL=... MONGODB_URI=... MAILER_DSN=... — les variables sensibles sont stockées côté serveur, jamais dans le code.
5. Déploiement	fly deploy — Fly.io construit l'image Docker, la déploie sur la machine virtuelle et redémarre l'application automatiquement.
6. Création de la base MySQL sur Clever Cloud	J'ai créé un add-on MySQL sur Clever Cloud (clever-cloud.com). Clever Cloud fournit les identifiants de connexion (hôte, port, nom de base, utilisateur, mot de passe) qui sont ensuite renseignés dans la variable DATABASE_URL côté fly.io via fly secrets set.
7. Import de la base de données	La base MySQL est initialisée via l'import du fichier ecf_vg.sql sur le serveur Clever Cloud, en utilisant un client MySQL ou l'interface d'administration fournie.

8. Vérification

Accès à <https://ecf-vg.fly.dev> pour vérifier le bon fonctionnement.
Consultation des logs via fly logs en cas d'erreur.

Configuration fly.toml (extrait)

```
app = 'ecf-vg'
primary_region = 'cdg'

[http_service]
  internal_port = 80
  force_https = true

[[vm]]
  memory = '256mb'
  cpu_kind = 'shared'
  cpus = 1
```

URL de production

Application	https://ecf-vg.fly.dev
-------------	---